

Ниже – «паспорт» сущности План (= Envelope/Bucket), который остаётся одним и тем же объектом во всей системе, но ведёт себя по-разному в зависимости от трёх-четырёх параметров. Модель проверена на все ранее перечисленные сценарии (микро, макро, прогнозы, слои, авто-распределение, инвентаризация, негативные балансы и т. д.) и не требует «магии» – всё реализуется обычными SQL/cron-джобами + немного бизнес-кода.

Таблица Plan ————— plan • id, user_id • type enum ACCUM (копим), BUFFER (подушка), SINK (расход), PASS-THROUGH (доход/транзит) • periodicity enum NONE | DAILY | WEEKLY | MONTHLY | YEARLY • rollover_policy enum KEEP (остаток сохраняем), RESET (переносим в родителя/НЗ), CAP (обрезаем до лимита) • auto_priority smallint (0–100, 0 = не автопополнять) • goal_amount numeric, nullable • goal_date date, nullable • floor_amount numeric, nullable - «минимум пессимизма» • parent_id FK plan.id, nullable • archived_at timestamptz, nullable (остальное: цвет, emoji, сортировка – несущественно для логики).

Почему только это: пяти полей (type, periodicity, rollover, priority, цели) хватает, чтобы смоделировать любые «карманные», «iPhone к ноябрю», «ипотека 30 лет», «зарплата», «нерезервируемый доход» и т. д.

2. Балансы

У плана «живут» три числовых состояния; их можно хранить в materialized-view или считать налёту.

balance_actual – $\sum \text{legs WHERE status} = \text{'ACTUAL'}$ (факт). balance_planned(t) – fact + $\sum \text{legs.status=PLANNED \& dt} \leq t$ (базовый прогноз). balance_scenario(t,s) – fact + planned + $\sum \text{legs.scenario_id=s}$ (опт/песс и т. д.). Для UI нужны именно time-series → nightly job прогоняет генератор stream_instance → заполняет plan_balance_daily(plan_id, dt, scenario_id, balance).

3. Как деньги попадают в план?

Источник всегда TxLeg(envelope_id = plan.id, amount_signed).

Леги бывают четырёх происхождений:

A. Ручные транзакции пользователя. B. Импорт (банковская выгрузка) + auto_link по категории. C. MoneyStream → генератор создаёт legs status=PLANNED. D. Engine-allocation («распирай свободный кэш»).

Правило D (auto-распределение) выполняется так:

Собираем все планы type=ACCUM|BUFFER с auto_priority>0, сортируем по приоритету DESC. Для каждого вычисляем need = goal_amount - balance_planned(goal_date). • если goal_amount null → need = floor_amount - balance_actual (для «подушки»). Проходим списком, пока есть свободный cash на счетах, создаём Tx с legs ±need. Алгоритм чисто серверный, работает раз в день или «по кнопке», поэтому система понятна и предсказуема (пользователь всегда видит, какую сумму и куда она залила).

4. Как деньги покидают план?

• type = SINK – generator разворачивает MoneyStream «-X» по periodicity; к нужной дате баланс → 0. – overspend разрешён (balance может уйти в минус).

- type = ACCUM / BUFFER – деньги снимаются только явной транзакцией (чек, перевод) или when rollover_policy=RESET/ CAP и наступил новый период. – если balance упал ниже floor_amount → UI подсвечивает «риск».
- type = PASS-THROUGH – зеркало для зарплаты/кэшбэка: на нём баланс всегда ≈ 0 , все входы сразу распределяются Engine-allocation'ом.

5. Периодичность и rollover

Пара (periodicity, rollover_policy) заменяет «бюджетные периоды», «сколько можно перенести» и т. д.

Примеры • «Карманные 3000/нед» → type=SINK, periodicity=WEEKLY, rollover=RESET (остаток стираем). • «Абонентка 600 Р/мес» → SINK / MONTHLY / KEEP (остаток копится, если заплатил меньше). • «Подушка 3*3П» → BUFFER / NONE / KEEP, goal_amount = X. • «Аванс 30 % зарплаты» → PASS-THROUGH / MONTHLY / RESET.

Сам переход периода – это cron-job: 1) вычислить new_period_start, 2) если rollover=RESET → TxLeg(-balance, plan) + TxLeg(+balance, parent_plan или UNASSIGNED).

6. Родитель-дети

• балансы не дублируются: balance_parent = $\sum$ children. (Tx нельзя написать одновременно в parent и child). • allocation_scope ограничивается одной «веткой» – это позволяет делать: – parent «Дом» (goal 1 млн) → children «Ремонт», «Мебель», «Техника». – Engine сначала закрывает цели детей, остаток льётся в родителя.

Edge-cases и решения а) child переполнился (balance > goal) – if parent.rollover=CAP → лишнее течёт в родителя (Tx «-over, +over»). б) parent имеет rollover=RESET, а дети – KEEP – разрешаем: при RESET деньги детей не трогаем.

7. Проверки консистентности

double-entry Ledger → $\sum(\text{account-legs}) = -\sum(\text{plan-legs}) = 0$. $\sum$ latest InventorySnapshot = $\sum$ account.balance_actual ($\pm$ копейки курсов). $\sum$ plan.balance_actual = 0 (в системе нет «ничейных» денег). Если 3 нарушено – остаток лежит на спец-плане UNASSIGNED (type=BUFFER, priority=0).

8. Что уже покрыто / где могут быть дыры?

✓ Микро-режим: баланс_actual, overspend-маркеры, weekly RESET. ✓ Макро-режим: плановые + сценарные ряды до n лет вперёд. ✓ What-If: scenario_id в legs. ✓ Негативные планы: только type=SINK (явно), отчёт 7 «красные». ✓ Поддержка «нерегулярных, но частых» – MoneyStream с RRULE «FREQ=DAILY;INTERVAL=3». X Плавающий курс валют. Решение: currency_code в Plan, план-балансы храним в валюте плана + view с пересчётом в базу. X Инфляционная индексация целей. Простое решение – раз в год скрипт «goal_amount *= 1.07» по отмеченным планам.

9. Почему использование остаётся

простым

Пользователь создаёт всего два объекта: «Купить MacBook 200 000 к ноябрю» → Plan(type=ACCUM, goal_amount, goal_date). «Откладывать 40 000/мес» → MoneyStream(+40 000/mo, envelope=Plan). Дальше Engine сам равномерно забрасывает деньги, UI показывает дату достижения.

Для «карманных» пользователь ничего не считает: Plan(type=SINK, periodicity=WEEKLY, amount=5000). Stream зарплата → Engine гонит деньги; при перерасходе Plan уходит в минус → красный фон.

«Подушка» держится автоматикой: приоритет=90, floor_amount=3*salary. Всё «свободное» заливается туда до floor.

Родитель/child строятся drag&drop – нет ручных сумм-связок.

Итог: одна таблица Plan + 4–5 простых правил дают:

- предсказуемое движение денег;
- отсутствие «долларов из воздуха»;
- минимальный набор полей при создании;
- поддержка любых отчётов и What-If-сценариев, которые вы описывали.